

PEFT: QLoRA, DoRA & AdaLoRA

Dhruv Kumar

BITS Pilani

What is quantization?

Quantization maps a continuous, high-precision number to one of a small, fixed set of representable values, storable in far fewer bits. With b bits you get exactly 2^b representable levels — **4 bits means 16** possible values for every weight.

$$x_{int} = \text{clamp}(\lfloor x/s \rfloor + z, 0, 2^b - 1), \quad \hat{x} = s(x_{int} - z)$$

x — the original value; s — the scale; z — the integer zero-point; b — the bit-width; x_{int} — the stored integer; $\hat{x}$ — the dequantized approximation, used whenever the weight is actually multiplied.

Calibrate once per tensor from its observed range $[x_{min}, x_{max}]$: $s = \frac{x_{max} - x_{min}}{2^b - 1}$, $z = \text{round}(-x_{min}/s)$ — so z is just the integer code that $x = 0$ maps to. With $b = 4$ and range $[-2, 6]$: $s = \frac{6 - (-2)}{15} = 0.533$, $z = \text{round}(2/0.533) = 4$:

x	$x/s + z$	x_{INT}	DEQUANTIZED $s(x_{INT} - z)$
-2.0	0	0	-2.133
0.0	4	4	0.000
3.2	10	10	3.200
6.0	15	15	5.867
9.0	21	15 (clamped)	5.867

WHY BOTHER, AND WHY CLAMP?

Bit-width sets memory directly: fp32 $\rightarrow$ 32 bits/weight, bf16 $\rightarrow$ 16, int8 $\rightarrow$ 8, int4 (like NF4) $\rightarrow$ 4 — a **4×** reduction going 16-bit to 4-bit. Clamping matters for the last row: **9.0** is outside the calibrated range, so $x/s + z = 21$ would need a 5th bit; clamp forces it to the nearest valid code (**15**) instead of overflowing — at the cost of a much larger error (**3.13** vs. ≤ 0.13 for in-range values).

QLoRA

QLoRA backpropagates LoRA's B, A gradients through a **frozen base model held in 4-bit precision** — this is what "quantization-aware LoRA" means in practice.

4-bit NormalFloat

A data type tuned to how pretrained weights are actually distributed (roughly normal), so 4 bits capture more information than a generic 4-bit float.

Double quantization

The quantization constants themselves are quantized again, shrinking memory further.

Paged optimizers

Optimizer states spill to CPU memory on demand, avoiding out-of-memory spikes.

HEADLINE RESULT

Fine-tunes a 65B-parameter model on a **single 48GB GPU**, matching full 16-bit fine-tuning quality.

NF4: where the 16 levels come from

Pretrained weights are roughly normally distributed, so NF4 places its 16 levels at the **quantiles** of a standard normal $N(0, 1)$ — each level covers an equal slice of *probability*, not an equal slice of the number line:

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k+1} \right) + Q_X \left(\frac{i+1}{2^k+1} \right) \right)$$

Q_X — the quantile function (inverse CDF) of $N(0, 1)$; $k = 4$ is the bit-width; q_i is the midpoint of two adjacent quantiles out of $2^k + 1 = 17$. Since $N(0, 1)$ has mean 0, Q_X is antisymmetric about $p = 0.5$, so this grid's exact-zero index $q_{2^{k-1}}$ only cancels to 0 if the two quantile evaluations round *identically* — fragile in floating point. NF4 sidesteps relying on that cancellation: build **two half-tables** at reduced resolution — 8 quantiles for the negative side, 9 for the positive side, each via the same formula — then merge them, sharing one endpoint fixed at exactly 0. Rescale by the largest $|q_i|$ so everything fits in $[-1, 1]$, giving the fixed, precomputed table every weight gets snapped to, next slide onward:

INDEX	VALUE	INDEX	VALUE
0000	-1.000	1000	0.080
0001	-0.696	1001	0.161
0010	-0.525	1010	0.246
0011	-0.395	1011	0.338
0100	-0.284	1100	0.441
0101	-0.185	1101	0.563
0110	-0.091	1110	0.723
0111	0.000	1111	1.000

THIS IS THE TABLE EVERY INDEX IN THIS LECTURE POINTS INTO

Index 0111 is exactly 0 — the payoff of the asymmetric merge. Index 1000 → 0.080 — hold onto that number; it's exactly the lookup used a couple of slides from now. Levels are packed tightly near zero and spread out at the extremes, because that's where a normal distribution puts most of its probability mass.

QLoRA: from weights to 4-bit codes

Real weights aren't already sitting in $[-1, 1]$ — NF4 has to get them there first. Weights are grouped into small **blocks** (64 weights per block in practice), and every block goes through the same four steps:

1. **Find the scale.** $s = \max_i |w_i|$ — the largest magnitude in the block.
2. **Normalize.** Divide every weight by s , so the whole block now lies in $[-1, 1]$: $\tilde{w}_i = w_i / s$.
3. **Snap to the nearest level.** Compare $\tilde{w}_i$ to the 16 NF4 quantile levels and keep only the **4-bit index** of the closest one.
4. **Store.** One 4-bit index per weight, plus a single shared scale s for the whole block.

GETTING A WEIGHT BACK — CONCRETELY

Say a weight's stored 4-bit index is `1000` (decimal 8). Look up index 8 in the fixed 16-value table $\rightarrow$ **0.080**. Multiply by the block's scale, say $s = 0.88$: $\hat{w} = 0.080 \times 0.88 = 0.070$. That's the number actually used in the matrix multiply — the original weight might have been **0.05**; the small gap is quantization error. This lookup-then-scale happens on the fly, right before each matrix multiply, never stored.

QLoRA: a worked example

Take a block of 8 weights (real blocks hold 64 — small here for clarity):

$$w = [0.05, -0.12, 0.31, -0.55, 0.02, 0.88, -0.03, 0.15]$$
$$s = \max_i |w_i| = 0.88$$

W_I	W_I/S	4-BIT INDEX	DEQUANTIZED
0.05	0.057	1000	0.070
-0.12	-0.136	0110	-0.080
0.31	0.352	1011	0.338
-0.55	-0.625	0001	-0.613
0.02	0.023	0111	0.000
0.88	1.000	1111	0.880
-0.03	-0.034	0111	0.000
0.15	0.171	1001	0.142

WHAT ACTUALLY GETS STORED

8 weights × 4 bits + one 32-bit scale = **64 bits** for this block — vs. $8 \times 16 = 128$ bits at fp16, already a 2× saving here, and real 64-weight blocks push this much further since one scale is shared over far more weights. Dequantizing recovers each weight only *approximately* — mean absolute error **0.024** — exactly the accuracy/memory trade QLoRA is making.

QLoRA: what's used where

Two different objects flow through QLoRA, stored and used differently:

BASE WEIGHT W_0

Stored in 4-bit, **always** — during fine-tuning and at inference alike. Every forward pass, each block is dequantized on the fly to bf16 right before its matmul, then discarded. W_0 never receives a gradient and is never updated.

LORA'S B, A

Kept in bf16 throughout — small enough that quantizing them wouldn't save much, and they need to accumulate gradients precisely. Trained during fine-tuning; frozen (but still applied) at inference.

Both stages run the **same** computation, $h = \text{dequant}(W_0)x + \frac{\alpha}{r}(xB)A$ — training additionally tracks gradients through it, into B, A only; inference just evaluates it.

MERGING IS POSSIBLE — IT JUST UNDOES THE POINT

Plain LoRA merges B, A into W_0 once, offline, before deployment — inference then runs a single matmul at zero added cost (an earlier slide). QLoRA *could* do the same: dequantize W_0 to bf16, add $\frac{\alpha}{r}BA$, store the result. But that merged model is bf16-sized again — it throws away the 4-bit memory saving that was the whole reason to quantize W_0 . So QLoRA keeps W_0 and B, A unmerged and pays for it on every forward pass: dequantizing each block on the fly, plus a separate small matmul, both of which a merged model would skip. This is genuinely extra latency versus plain LoRA — the trade for a much smaller resident model.

QLoRA: NF4 in action

Quantize 5 sample weights (rescaled into $[-1, 1]$) with NF4 levels — denser near zero — vs. plain uniform 4-bit levels:

WEIGHT X	NF4 DEQUANTIZED	UNIFORM DEQUANTIZED
0.05	0.080	0.067
-0.12	-0.091	-0.067
0.31	0.338	0.333
-0.55	-0.525	-0.6
0.88	1.0	0.867

FIVE POINTS AREN'T ENOUGH TO TRUST

On just these 5 points, NF4's mean error (**0.046**) is actually *higher* than uniform's (**0.031**) — the outlier at 0.88 lands right on an NF4 edge level and gets amplified. NF4's real advantage only shows up in *expectation*, over many weights.

AVERAGED OVER 20,000 SIMULATED 64-WEIGHT BLOCKS

Drawing each block from a standard normal — matching how pretrained weights are actually distributed — mean absolute error is **0.073** for NF4 vs. **0.085** for uniform: **14.7% lower**. This is what "NF4 is information-theoretically optimal for normal data" actually means — a population-level guarantee, not a promise for any single block.

QLoRA: double quantization in numbers

Every block of **64** weights shares one scale constant s , normally stored in 32-bit float — that's a real, per-parameter memory cost on top of the 4-bit weights themselves:

$$\text{overhead} = \frac{32 \text{ bits}}{64 \text{ weights}} = 0.5 \text{ bits/weight}$$

Double quantization quantizes those scale constants too: every 256 of them (themselves covering $256 \times 64 = 16,384$ weights) share one 32-bit meta-constant, and each individual scale is stored in only 8 bits:

$$\text{overhead} = \frac{256 \times 8 \text{ bits} + 32 \text{ bits}}{16,384 \text{ weights}} = \frac{2080}{16,384} \approx 0.127 \text{ bits/weight}$$

AT MODEL SCALE

$0.5 - 0.127 = 0.373$ bits saved per weight. Across a 65-billion-parameter model, that's $0.373 \times 65\text{B}$ bits $\approx$ **3.0 GB** of GPU memory recovered — from compressing numbers that were only ever there to describe *other* numbers.

QLoRA: paged optimizers

Adam's optimizer state (m , v per trainable parameter) is small for LoRA's B , A — but GPU memory use isn't perfectly flat during training. Operations like gradient-checkpoint recomputation cause short-lived **spikes** above the steady-state usage.

WITHOUT PAGING

The GPU must be provisioned for the *peak*, not the average. Suppose steady-state training fits in 20GB but one recomputation step briefly needs 24GB — on a 22GB GPU, that spike alone crashes the run, even though it happens for a fraction of a second.

WITH PAGING

Paged optimizers use NVIDIA unified memory: pages of optimizer state not needed *right now* are automatically migrated to CPU RAM when the GPU is under pressure, and paged back in when needed — the same idea an operating system uses to page virtual memory to disk. The 22GB GPU survives the 24GB spike by briefly borrowing 2GB of CPU RAM, then returns to normal.

DoRA: weight-decomposed LoRA

DoRA splits a weight matrix into two independent pieces — a **magnitude** and a **direction** — and applies LoRA only to the direction.

$$W = m \cdot \frac{V}{\|V\|_c}$$

V — the direction matrix, initialized to W_0 ; $\|V\|_c$ — its column-wise norm, so $V/\|V\|_c$ has unit-norm columns; m — a separately trained magnitude vector, one scalar per column, initialized to $\|W_0\|_c$.

LoRA's update is applied only inside the direction term: $V = W_0 + BA$. Training m and B, A separately lets the model adjust a column's *length* and its *orientation* independently — closing more of the gap to full fine-tuning than plain LoRA, at no extra inference cost.

DoRA: a numeric example

Reuse LoRA's W_0 and update: $V' = W_0 + \frac{\alpha}{r}BA = \begin{bmatrix} 4 & 5 \\ 2 & 5 \\ 1 & 2 \end{bmatrix}$, column norms $\|V'\|_c \approx [4.58, 7.35]$.

If m were left passive at $\|V'\|_c$, DoRA would just reproduce plain LoRA's W' . Suppose training instead finds an **independent** $m_{\text{learned}} = [5.0, 6.5]$:

$$W_{\text{DoRA}} = m_{\text{learned}} \cdot \frac{V'}{\|V'\|_c} = \begin{bmatrix} 4.36 & 4.42 \\ 2.18 & 4.42 \\ 1.09 & 1.77 \end{bmatrix} \quad \text{vs. plain LoRA's} \quad W' = \begin{bmatrix} 4 & 5 \\ 2 & 5 \\ 1 & 2 \end{bmatrix}$$

SAME B, A – DIFFERENT WEIGHT

The direction update is identical to plain LoRA's, but the final weight matrix visibly differs, because m was free to land somewhere other than the column norm BA would have produced on its own. That extra degree of freedom is what closes more of the gap to full fine-tuning.

AdaLoRA: adaptive rank allocation

Plain LoRA gives every weight matrix the **same fixed rank** r . AdaLoRA instead parameterizes each update in SVD form and lets the rank vary by importance.

$$\Delta W \approx P \Lambda Q$$

$P \in \mathbb{R}^{d \times r}$, $Q \in \mathbb{R}^{r \times d_{out}}$ — like LoRA's B , A , but regularized to stay near-orthonormal; $\Lambda = \text{diag}(\sigma_1, \dots, \sigma_r)$ — singular-value-like weights, one per rank triplet.

During training, each triplet (p_i, σ_i, q_i) gets an importance score; low-importance triplets are pruned ($\sigma_i \rightarrow 0$) and the freed rank budget is reallocated to more important matrices — so more useful layers end up with higher effective rank than less useful ones, for the same total parameter budget.

AdaLoRA: why SVD

Take $M = \begin{bmatrix} 3 & 1 & 2 \\ 2 & 4 & 1 \\ 1 & 1 & 5 \end{bmatrix}$, singular values $\sigma = [6.78, 3.56, 1.82]$. Keeping only the largest, $r = 1$:

$$M_1 = \sigma_1 u_1 v_1^\top, \quad \|M - M_1\|_F = 4.00$$

Compare against a different, non-SVD rank-1 guess — keep only the top-left entry, zero everything else:

$$M_{\text{arb}} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \|M - M_{\text{arb}}\|_F = 7.28$$

ECKART-YOUNG, CONFIRMED

$4.00 < 7.28$ — truncated SVD's error is smaller, and it's provably the **smallest possible** among every rank-1 matrix, not just this one alternative. That's why AdaLoRA parameterizes its update in SVD form: the singular values say exactly which rank-1 pieces are safest to drop.

Reading list

THIS LECTURE'S PAPERS

[Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs" \(2023\)](#)

[Liu et al., "DoRA: Weight-Decomposed Low-Rank Adaptation" \(2024\)](#)

[Zhang et al., "AdaLoRA: Adaptive Budget Allocation for PEFT" \(2023\)](#)

SELF-STUDY

ReLoRA repeatedly merges and restarts LoRA updates during training, approximating a higher effective rank over time.